

Reinforcement Learning

Intelligent Systems II

Mário Antunes

Universidade de Aveiro

2026

Part I: Foundations of Reinforcement Learning

Beyond Supervised Learning

- **Supervised Learning:** Learning from a “Teacher” who provide the correct labels y for every input x .
- **Reinforcement Learning (RL):** Learning from a “Critic” who only provides a scalar **Reward** signal R .
- **The Challenge:**
 - **No Labels:** The agent is never told which action was “correct.”
 - **Delayed Feedback:** An action now might only yield a reward many steps later (Credit Assignment Problem).
 - **Exploration:** The agent must try new things to find rewards.

The RL Interaction Loop

An agent exists in a closed loop with its environment.

1. **Sense:** Agent perceives current state s_t .
2. **Act:** Agent executes action a_t based on its policy π .
3. **Transition:** Environment moves to s_{t+1} with probability $P(s_{t+1}|s_t, a_t)$.
4. **Reward:** Agent receives scalar r_{t+1} .

Analogy: Training a dog. You don't "program" the dog's muscles; you provide a treat (Positive Reward) when it sits, and it must figure out which muscle movements led to the treat.

A Reinforcement Learning task is formally defined as a **Markov Decision Process (MDP)**.

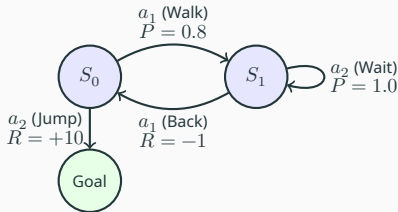
The Tuple $\{S, A, P, R, \gamma\}$:

- S : Set of all possible states.
- A : Set of all possible actions.
- $P(s'|s, a)$: Transition probability (Dynamics).
- $R(s, a, s')$: Reward function.
- $\gamma \in [0, 1]$: Discount factor (Impatience).

Formal Specification: The MDP ii

Scenario: The Slippery Bridge

An agent starts at S_0 . It seeks to reach the **Goal**. Actions are stochastic: walking (a_1) might fail, and waiting (a_2) keeps the state.



- **Dynamics (P):** Walking from S_0 has an 80% chance to reach S_1 .
- **Rewards (R):** Reaching the Goal gives $+10$; backtracking costs -1 .
- **Transitions:** Arrows represent the probability of s_{t+1} given (s_t, a_t) .

The Goal: Expected Return

The agent's objective is to maximize the **Cumulative Discounted Reward**, called the **Return** (G_t).

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

- **Why γ ?**

1. **Math:** Ensures the sum converges for infinite horizons.
2. **Uncertainty:** Future rewards are less certain than immediate ones.
3. **Animal Behavior:** Preferred "now" over "later."

The State-Value Function $V(s)$

How “good” is it to be in a particular state?

Definition: The expected return starting from state s and following policy π .

$$V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$$

- $V(s)$ represents the long-term value, whereas r represents the immediate payoff.
- A state with $r = 0$ might have a high $V(s)$ if it leads to a goal.

The Action-Value Function $Q(s, a)$

How “good” is it to take a specific action in a state?

Definition: The expected return starting from state s , taking action a , and then following policy π .

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$$

- **The Optimal Policy:** $\pi^*(s) = \arg \max_a Q^*(s, a)$.
- If we know $Q^*(s, a)$, the problem is solved: we always pick the action with the highest Q .

The Bellman Equation

The fundamental recursive relationship of RL.

$$V(s) = \mathbb{E}[r_{t+1} + \gamma V(s_{t+1}) | s_t = s]$$

Deterministic Form:

$$V(s) = R(s, a) + \gamma V(s')$$

Insight: The value of the current state is the immediate reward plus the discounted value of the next state. This allows us to “spread” the reward from the goal back to the starting position.

Step-by-Step Example: 1D Walk i

Environment:

- 3 Cells: $[A, B, C]$.
- Start at A . Goal at C .
- Actions: Right (Deterministic).
- Reward: $r = 0$ for $A \rightarrow B$, $r = 10$ for $B \rightarrow C$.
- Parameters: $\gamma = 0.9$.

Initial Values: $V(A) = 0, V(B) = 0, V(C) = 0$.

Step-by-Step Example: 1D Walk ii

Iteration 1 (Update B):

$$V(B) = R(B, \textit{Right}) + \gamma V(C) = 10 + 0.9(0) = 10$$

Iteration 2 (Update A):

$$V(A) = R(A, \textit{Right}) + \gamma V(B) = 0 + 0.9(10) = 9$$

Final Values: $V(A) = 9$, $V(B) = 10$.

The agent now “knows” that A is valuable because it leads to the 10 reward in B , even though A itself gives no reward.

Part II: Q-Learning

From Model-Based to Model-Free

- **Model-Based:** Agent knows $P(s'|s, a)$ and R . (Like solving a puzzle with the manual).
- **Model-Free:** Agent does **not** know how the world works. It must learn through trial and error.

Q-Learning is a model-free algorithm that learns $Q(s, a)$ directly from experience tuples (s, a, r, s') .

The Q-Learning Update Rule

Based on **Temporal Difference (TD) Learning**.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{Target}} - \underbrace{Q(s, a)}_{\text{Current}} \right]$$

- $\alpha \in (0, 1]$: Learning rate (How much to trust new info).
- **TD Error**: The difference between what we thought Q was and the new “Target” we just observed.

Grid World Setup

	S_2	GOAL +10
S_1	PIT -10	
START		

- S_1, S_2 : Intermediate states used in our update examples.
- **Rewards**: +10 for reaching the Goal; -10 for falling into the Pit.
- **Goal**: Terminal state where the episode ends.
- **Task**: Learn the shortest path from START to GOAL while avoiding PIT.

Step: Agent is at S_2 (top center). Takes action *Right* to reach Goal.

- Observed: $r = 10, s' = Goal$.

Update:

$$Q(S_2, R) \leftarrow 0 + 0.5[10 + 0.9(0) - 0] = 5$$

- *Note:* $\max_{a'} Q(Goal, a')$ is 0 because the episode ends.

Grid World: Iteration 2

Step: Agent is at S_1 (left center). Takes action Up to reach S_2 .

- Observed: $r = 0, s' = S_2$.

Update:

$$Q(S_1, U) \leftarrow 0 + 0.5[0 + 0.9(\max_{a'} Q(S_2, a')) - 0]$$

$$Q(S_1, U) \leftarrow 0.5[0.9(5)] = 2.25$$

The value is propagating from the goal backwards!

Exploration vs. Exploitation

If the agent always picks $\arg \max Q$, it might get stuck in a “good enough” path and never find the “optimal” one.

ϵ -greedy Policy:

- With probability $1 - \epsilon$: **Exploit** (Pick best known action).
- With probability ϵ : **Explore** (Pick random action).

***Strategy:** Start with $\epsilon = 1.0$ (pure exploration) and decay it over time as the agent becomes more confident.*

Part III: Q-Learning Variants

Update Rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$$

- **Difference:** Q-Learning assumes we will take the *best* next action (max). SARSA uses the *actual* next action a' chosen by the policy.
- **Analogy:** Q-Learning is an optimist (assumes perfect future); SARSA is a realist (accounts for its own mistakes/exploration).

The Problem: Maximization Bias.

- Because Q values are noisy estimates, $\max Q(s', a')$ often picks an overestimated value.
- This leads to the agent becoming overconfident in bad actions.

The Solution: Use two independent Q-tables (Q_1 and Q_2).

Algorithm:

1. With 50% probability, update Q_1 using Q_2 for the estimate:

$$Q_1(s, a) \leftarrow Q_1 + \alpha [r + \gamma Q_2(s', \arg \max_{a'} Q_1(s', a')) - Q_1]$$

2. Otherwise, update Q_2 using Q_1 .

Logic: One table chooses the best action, the other evaluates it. They act as a “check and balance” system.

Part IV: Deep Reinforcement Learning

The Limits of Tabular RL

- **State Space Explosion:** In a 10×10 grid, we have 100 states. In an Atari game (210×160 pixels), we have $256^{210 \times 160}$ states.
- **The Curse of Dimensionality:** We cannot store a Q -table for high-dimensional or continuous inputs.
- **Generalization:** If we learn $Q(s, a)$ for one state, we want that knowledge to apply to “similar” states.

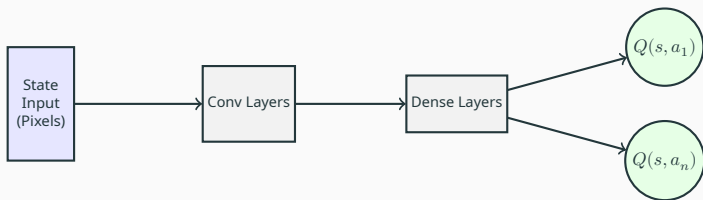
The Solution: Use a **Function Approximator** (Neural Network) to estimate $Q(s, a; \theta)$.

Formal Specification: Instead of a lookup table, we use a parameterized function:

$$Q(s, a) \approx Q(s, a; \theta)$$

- **Input:** State representation (e.g., raw pixels or sensor vectors).
- **Output:** A vector of Q -values, one for each possible discrete action.
- **Goal:** Find parameters θ that minimize the Bellman error.

Deep Q-Network (DQN) Architecture



- **Shared Representation:** The network extracts features that are useful for evaluating *all* actions simultaneously.
- **Efficiency:** A single forward pass gives us Q for every action.

The DRL Objective Function

We define a **Loss Function** based on the Mean Squared Bellman Error (MSBE).

$$J(\theta) = \mathbb{E}_{s,a,r,s'} \left[\left(\underbrace{y}_{\text{Target}} - \underbrace{Q(s,a;\theta)}_{\text{Prediction}} \right)^2 \right]$$

Where the target y comes from the Bellman equation:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta)$$

Problem: The target y depends on the same parameters θ we are trying to optimize. This is like “chasing your own tail.”

Policy Gradients: A Brief Intro

Instead of learning Q , why not learn the policy $\pi(a|s; \theta)$ directly?

The REINFORCE Objective:

$$J(\theta) = \mathbb{E}_{\pi}[G_t]$$

The Gradient:

$$\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi(a_t|s_t; \theta) G_t]$$

- **Intuition:** Increase the probability of actions that lead to high returns (G_t) and decrease it for low returns.
- **Note:** We will focus on **DQN** (Value-based) for the following implementation steps.

Calculating the Gradient: Step-by-Step i

Scenario: A single transition (s, a, r, s') .

1. **Forward Pass:** Compute $Q(s, a; \theta)$.
2. **Target Calculation:** Compute $y = r + \gamma \max_{a'} Q(s', a'; \theta)$.
3. **Loss:** $L = (y - Q(s, a; \theta))^2$.

Chain Rule for Weights w :

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial Q} \cdot \frac{\partial Q}{\partial w}$$

$$\frac{\partial L}{\partial Q} = -2(y - Q(s, a; \theta))$$

Calculating the Gradient: Step-by-Step ii

Example:

- $Q(s, a; \theta) = 5.0$
- $y = 7.0$ (Observed reward + discounted next state)
- Difference: 2.0

Update:

$$\theta \leftarrow \theta + \eta \cdot \underbrace{(y - Q(s, a; \theta))}_{\text{Error}} \cdot \nabla_{\theta} Q(s, a; \theta)$$

We push the weights in the direction that makes $Q(s, a)$ closer to y .

Part V: Deep Q-Learning (DQN)

Why Simple DRL Fails

Standard Deep Learning assumes **Independent and Identically Distributed (IID)** data. RL violates this: 1. **Correlated Samples:** Consecutive states (s_t, s_{t+1}, s_{t+2}) are highly similar. 2. **Non-Stationary Targets:** Updating θ changes the targets y for future steps, leading to oscillations or divergence.

The Solution: Mnih et al. (2015) introduced **DQN** with two key innovations.

Innovation 1: Experience Replay

Instead of learning from the current transition, store it in a **Replay Buffer** $\mathcal{D}$.

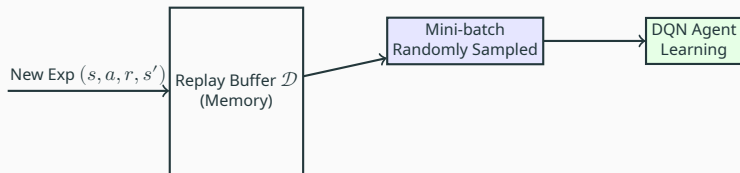
Algorithm:

1. Collect (s_t, a_t, r_t, s_{t+1}) and store in $\mathcal{D}$.
2. **Sample** a random mini-batch of transitions from $\mathcal{D}$.
3. Update θ using this batch.

Benefits:

- **Breaks Correlations:** Random sampling makes the data look more like IID.
- **Data Efficiency:** Each experience can be used for multiple updates.

Experience Replay Visualized



Innovation 2: Target Networks

To fix the “moving target” problem, use two networks: 1.

Q-Network (θ): The one being updated. 2. **Target Network**

(θ^-): A frozen copy used ONLY to calculate y .

The Target becomes:

$$y = r + \gamma \max_{a'} Q(s', a'; -)$$

- **Update:** Every C steps, synchronize the weights:
 $\theta^- \leftarrow \theta$.
- **Result:** Stable targets for C steps, allowing the network to converge.

Repeat for each episode:

1. For each step t in environment:
 - Select a_t using ϵ -greedy policy based on $Q(s_t, a; \theta)$.
 - Execute a_t , observe r_t, s_{t+1} .
 - Store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$.
 - **Sample** mini-batch from $\mathcal{D}$.
 - Calculate **Loss**: $L = \frac{1}{N} \sum (y_j - Q(s_j, a_j; \theta))^2$.
 - Perform **Gradient Descent** on θ .
 - Every C steps, update $\theta^- = \theta$.

Step-by-Step Update: Iteration 1

Setup:

- Mini-batch size $N = 1$.
- Experience sampled: $(s, a = \textit{Right}, r = 1, s')$.
- Parameters: $\gamma = 0.9, \eta = 0.01$.

Current Network Outputs:

- $Q(s, \textit{Right}; \theta) = 0.2$ (Our current guess).
- Target Network $Q(s', a'; \theta^-) = [0.5, 0.8, 0.3]$ for actions $[U, D, L]$.

Step-by-Step Update: Calculation

1. Find Max Next Q:

$$\max_{a'} Q(s', a'; \theta^-) = 0.8$$

2. Compute Target y :

$$y = 1 + 0.9(0.8) = 1.72$$

3. Compute TD Error:

$$\delta = y - Q(s, \text{Right}; \theta) = 1.72 - 0.2 = 1.52$$

4. Weight Update:

$$\theta_{new} = \theta_{old} + 0.01 \cdot 1.52 \cdot \nabla_{\theta} Q(s, \text{Right}; \theta)$$

The prediction 0.2 will be nudged towards 1.72.

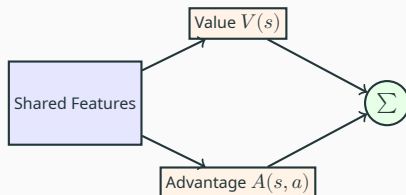
Part VI: DQN Variants

Observation: For many states, it doesn't matter which action you take (e.g., in a car, if there's no obstacle, any "forward" action is fine).

Solution: Decompose $Q(s, a)$ into two streams: 1. **Value Stream** $V(s)$: The value of being in state s . 2. **Advantage Stream** $A(s, a)$: How much better action a is compared to others.

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + A(s, a; \theta, \alpha)$$

Dueling Architecture Visualized



- **Benefit:** Learns the state value $V(s)$ more efficiently without needing to learn Q for every specific action individually.

Prioritized Experience Replay (PER)

Observation: Not all experiences are equally useful. Learning from a “lucky” goal or a “fatal” mistake is more important than wandering in an empty room.

Mechanism: 1. Assign a priority p_i to each transition based on its **TD Error**: $p_i = |\delta_i| + \epsilon$. 2. Sample transitions with probability proportional to p_i . 3. **Bias Correction:** Use Importance Sampling weights to ensure the update remains unbiased.

- **Result:** Faster convergence by focusing on the “surprising” transitions.

Double Deep Q-Learning (DDQN)

Just like in tabular RL, DQN suffers from overestimation.

DQN Update:

$$y = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta^-); \theta^-)$$

DDQN Update:

$$y = r + \gamma Q(s', \arg \max_{a'} Q(s', a';); \theta^-)$$

- Use the **Online Network** (θ) to select the best action.
- Use the **Target Network** (θ^-) to evaluate that action.
- This decouples the selection from the evaluation, reducing bias.

Part VII: Neuroevolution

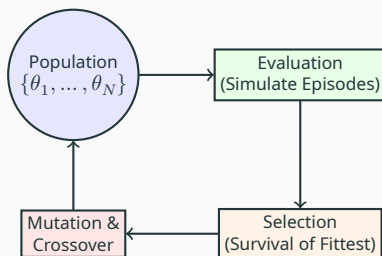
Definition: Using **Evolutionary Algorithms (EA)** to optimize the parameters (θ) or the topology of a Neural Network.

- **Mechanism:** Instead of calculating ∇J , we maintain a **Population** of agents and use selection pressure to improve performance.
- **Policy Search:** We treat the RL problem as a Black-Box optimization:

$$\theta^* = \arg \max_{\theta} F(\theta)$$

- **Fitness Function $F(\theta)$:** The total return G_0 obtained by an agent in an episode.

The Neuroevolution Loop



- **Parallelism:** Since evaluations are independent, Neuroevolution scales linearly with available CPU cores.
- **Stochasticity:** Naturally handles non-differentiable rewards or noisy environments.

1. Selection: Probability of an individual θ_i being selected for the next generation:

$$P(\theta_i) = \frac{\exp(F(\theta_i)/\tau)}{\sum_j \exp(F(\theta_j)/\tau)}$$

- τ : Temperature parameter (controls selection pressure).

2. Mutation: Perturbing the weights with Gaussian noise:

$$\theta_{child} = \theta_{parent} + \mathcal{N}(0, \sigma^2)$$

- σ : Mutation power (Step size).

3. Crossover (Recombination): Combining weights from two parents θ_A and θ_B :

$$\theta_{child} = \alpha\theta_A + (1 - \alpha)\theta_B$$

- $\alpha \in [0, 1]$: Mixing coefficient (can be a vector for per-weight mixing).

Advantages for Agents:

- Can optimize **discrete** parameters (e.g., number of neurons).
- Does not require a differentiable “World Model” or a reward gradient.
- Robust to “Deceptive Rewards” (where a local peak hides the global optimum).

Step-by-Step Generational Update

Generation t :

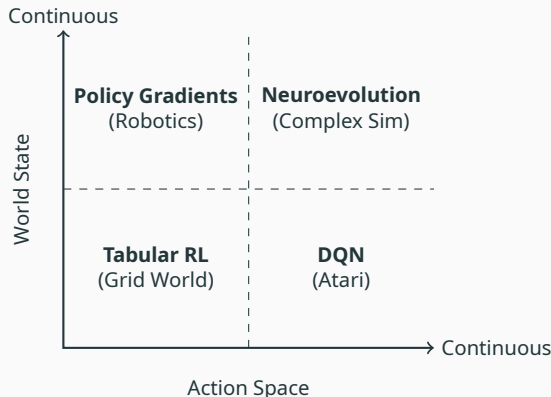
1. **Initial Pop:** $\{\theta_1, \theta_2\}$ with fitness $F_1 = 10, F_2 = 50$.
2. **Selection:** θ_2 is chosen as the “Elite” (higher fitness).
3. **Reproduction:**
 - $\theta_{2A} = \theta_2 + \mathcal{N}(0, 0.1)$ (Mutant A)
 - $\theta_{2B} = \theta_2 + \mathcal{N}(0, 0.1)$ (Mutant B)
4. **Generation $t + 1$:** $\{\theta_{2A}, \theta_{2B}\}$ replaces the old population.

Result: *The entire population shifts towards the high-fitness region of the weight space.*

Part VIII: Connection to AI Agents

Mapping Environments to Models

The choice of RL algorithm depends on the **dimensions of the environment** (revisited from Slide 01).



- **Discrete/Discrete:** Exact solutions (Q-Tables).
- **Continuous/Continuous:** Function approximation or global search.

RL in the Agent Lifecycle

How RL fits into the “Sense-Decide-Act” loop of a rational agent.

1. **Passive Agent:** Learns from observation ($V(s)$ only).
2. **Active Agent:** Learns from action ($Q(s, a)$).
3. **Reflex Agent:** Uses learned $\pi(a|s)$ as a direct lookup.
4. **Goal-Based Agent:** Uses $Q(s, a)$ as a “Heuristic” for planning.

The Hybrid Edge: Modern agents use **Deep RL** for perception (understanding the world) and **Logic/Search** for high-level safety constraints.

Architecture: Combining Deep RL with Monte Carlo Tree Search (MCTS).

1. **Policy Network:** Trained via Behavioral Cloning (expert games) then improved via Self-Play RL.
 2. **Value Network:** Trained via RL to predict who will win from a given board state.
 3. **MCTS:** Uses the networks to prune the search tree, focusing only on “promising” moves.
- **Impact:** Proved that AI can beat humans in high-complexity intuition-based games.

Task: Walking, grasping, or flying.

- **The Sim-to-Real Gap:** Training a physical robot is slow and dangerous. Agents are trained in **Physics Simulators** ([Isaac Gym](#), [MuJoCo](#)).
- **Domain Randomization:** Varying gravity, friction, and mass during training so the agent learns to be robust.
- **Result:** Agents that can walk over rubble or recover from kicks (Boston Dynamics style control).

- **Reinforcement Learning** (RL) is the science of learning optimal behavior from trial and error through an interaction loop with an environment.
- **Markov Decision Processes** (MDPs) provide the formal framework, where the **Bellman Equation** defines the recursive relationship of value functions.
- **Q-Learning** enables model-free learning by updating action-value estimates based on temporal difference errors.
- **Deep RL** (DQN) scales these concepts to high-dimensional state spaces using neural networks as function approximators.

- **DQN Stability:** Achieved through **Experience Replay** (breaking correlations) and **Target Networks** (stabilizing targets).
- **Advanced Architectures:** Dueling DQN, PER, and Double DQN address efficiency, learning priority, and overestimation bias.
- **Neuroevolution:** Provides a gradient-free alternative, treating parameters as genotypes and return as fitness.
- **AI Agents:** Leverage RL to navigate discrete and continuous environments, often hybridizing perception with logical reasoning.

Further Materials

Textbooks:

- Sutton & Barto, [Reinforcement Learning: An Introduction](#) (The “Bible”).
- Russell & Norvig, [AI: A Modern Approach](#) (Chapter 21).

Online Courses:

- [DeepMind x UCL Reinforcement Learning Lecture Series](#).
- [Hugging Face Deep RL Course](#) (Hands-on with Gymnasium).

Tools:

- [Gymnasium \(OpenAI Gym\)](#): Standard API for environments.
- [Stable Baselines3](#): Reliable Pytorch implementations of DQN, PPO, etc.
- [Ray RLLib](#): Scaling RL to massive clusters.

End of Module 09

Questions?